

Final Assignment
Cryptography and Cyber Law

Nazibur Rahman

IT-21017

Question 1: How does Shor's algorithm threaten the security of RSA and Elliptic Curve Cryptography (ECC), and what are the potential consequences for current digital infrastructure?

Ans: Shor's algorithm can efficiently solve the two math problems that are RSA and ECC rely on: integer factorization and the discrete logarithm problem. So, a large, fault-tolerant quantum computer running Shor's would completely break the mathematical basis of those systems. That would undermine nearly all widely used public-key crypto used for confidentiality, authentication, and integrity.

How Shor's algorithm attack RSA and ECC:

⇒ RSA - Integer factoring

RSA's security depends on the practical difficulty of factoring a large composite number $N = p \cdot q$. Shor's algorithm finds the prime factors in polynomial time on a quantum computer, so it would recover the private key from a public modulus.

⇒ ECC - discrete logarithm

ECC security depends on the elliptic-curve discrete logarithm problem (ECDLP): given a public point $Q=kP$, it's hard to find scalar k . Shor's algorithm also efficiently solves discrete-log problems (including ECDLP), so, it would recover ECC private keys from public keys.

What would be affected (real-world consequences)

Shor's would cripple almost every system that uses public-key cryptography for key exchange, signatures, or authentication. Examples:

- TLS/HTTPS and VPNs: Passive eavesdroppers who record encrypted traffic could decrypt sessions if they can derive session keys or if long-term keys are exposed. Active attacks that impersonate servers/clients via forged signatures, also become possible.
- Email and document confidentiality: Email encryption and archived encrypted documents could be decrypted.
- Code Signing and Software updates: Attackers could forge signatures and distribute malicious updates that appear authentic.
- Authentication and identity (PKI): Certificate authorities, code-signing certificates, client certs - the whole X.509

ecosystem would be at risk; attackers could impersonate service of users.

• Secure messaging and end-to-end encryption: If long-term private keys are exposed, messages can be forged or previous messages decrypted.

Question 2: Discuss the role of quantum key distribution in future cryptographic systems. How does it differ from classical public-key encryption?

Ans:

Quantum Key Distribution (QKD) is a method of securely sharing encryption keys using the laws of quantum mechanics. It relies on principles like the no-cloning theorem and measurement disturbance.

In future cryptographic systems, QKD can provide information-theoretic security; meaning its protection does not depend on the computational power of an attacker. Even a quantum computer cannot break it without being detected. This makes it valuable for securing sensitive communications in government, military, financial, and critical infrastructure.

How QKD differs from classical public-key encryption

Feature	Classical Public-key Encryption	Quantum Key Distribution
Security Basis	Computational hardness of math problems (factoring, discrete logs, lattice problems)	Fundamental laws of quantum physics (no-cloning, measurement disturbance)
Vulnerability of quantum computers	Breakable by Shor's algorithm (RSA/ECC) if a large quantum computer exists.	No breakable by quantum computers - eavesdropping changes the quantum states and can be detected.
Type of Cryptography	Asymmetric (Public/Private keys)	Symmetric Key distribution (Produces a shared key, but no encryption itself)
Key changes Channel	Purely classical network	Quantum channel for Photon transmission and classical channel for coordination

Question 3: What are the main differences between lattice-based cryptography and traditional number-theoretic approaches like RSA, particularly in the context of quantum resistance?

Ans:

Differences between lattice-based cryptography and RSA in the context of quantum resistance is given below:

Aspect	RSA (Traditional Number-Theoretic)	Lattice-Based Cryptography
Security basis	Hardness of integer factorization	Hardness of lattice problems, (e.g., Learning with errors, Shortest vector problem)
Quantum Resistance	Broken by Shor's algorithm on a large quantum computer	Believed secure against known quantum attacks
Mathematics used	Modular arithmetic over large integers.	High-dimensional integer lattices with noise terms.
Efficiency	Small public keys, but slow for large key sizes.	Larger public keys, often faster operations.
Applications	Will be phased out in the post-quantum era.	Encryption, digital signatures, key changes, plus advanced schemes.

Question 4: Develop a Python-based PRNG that uses the current system time and a custom seed value. Write a complete program and corresponding output.

Source Code:

```
import time

class SimpleLCG:
    def __init__(self, seed):
        self.state = seed
        self.A = 6364136223846793005
        self.C = 14426950408889663407
        self.MASK = (1 << 64) - 1

    def next_uint64(self):
        self.state = (self.state * self.A + self.C) & self.MASK
        return self.state

    def random(self):
        return (self.next_uint64() >> 11) / float(1 << 53)

    def splitmix64(n):
        n = (n + 0x9E3779B97F4A7C15) & ((1 << 64) - 1)
        n1 = (n >> 30); n = (n * 0xBF58476D1CE4E5B9) & ((1 << 64) - 1)
        n1 = (n >> 31)
        return n & ((1 << 64) - 1)
```


Main Program

Custom_seed = 12345

time_rs = time.time() - rs()

combined = (custom_seed + time_rs) & ((1 << 64) - 1)

init_state = Splitmix64(combined)

Prng = SimpleLCGz(init_state)

Print("First 5 random integers (0-100):")

for _ in range(5):

Print(Prng.next_uint64() % 101)

Print("\n First 5 random floats [0,1):")

for _ in range(5):

Print(Prng.random())

Output:

First 5 random integers (0-100):

19

62

1

4

55

First 5 random floats [0,1):

0.5702718976839399

0.5669494363405470

0.8037492119514423

0.2882359149945363

0.29342663922768290

Question-5: Explain the Sieve of Eratosthenes algorithm and use it to find all Prime Numbers less than 50. How does its time complexity compare to trial division?

Ans:

Sieve of Eratosthenes Algorithm

1. The sieve of Eratosthenes is an efficient method to find all prime numbers up to a given number n .
2. Start by creating a list of boolean values representing numbers from 2 to n , all initialized to True.
3. Begin with the first prime number = 2.
4. Mark all the multiples of p (i.e., $(2p, 3p, 4p)$) as False.
5. Find the next number greater than p that is still True, set p to this number, and repeat step 4.
6. Continue until $p^2 > n$.
7. The numbers remaining True in the list are the primes less than or equal to n .

Primes less than 50 (using the sieve)

- Numbers: 2 to 50
- Remove multiples of 2: 4, 6, 8, ... 50.
- Remove multiples of 3: 6, 9, 12, ... 48.
- Remove multiples of 5: 10, 15, 20, ... 50.
- Remove multiples of 7: 14, 21, 28, 35, ... 49.
- Stop after 7 since $7^2 = 49 \leq 50$

Primes under 50: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47

Time Complexity Comparison:

- Sieve of Eratosthenes: Runs in $O(n \log \log n)$ time. It marks multiples efficiently and only up to $\sqrt{n}$.
- Trial Division: To check each number up to n for Primality by dividing by all smaller primes up to $\sqrt{n}$, it takes about $O(n\sqrt{n})$ time.

$$(a \bmod n) \equiv a - \lfloor \frac{a}{n} \rfloor n$$

Necessary and Sufficient conditions (Kronecker's criterion)

If composite number n is a Carmichael number it satisfies only if all the following hold:

1. n is square-free, i.e. no prime factor appears with power greater than 1.

2. n is composite.

3. For every prime divisor p of n , $p-1$ divides $n-1$.

Formally,

For all primes $p|n$, $p-1|n-1$.

Question-6: State and explain the necessary and sufficient conditions for a composite number to be a Carmichael number. Then verify whether the number $n=561$ and $n=1105$ and $n=1729$ are Carmichael number?

Ans:

A Carmichael number is a composite number n that satisfies Fermat's little theorem for every integer a coprime to n :

$$a^{n-1} \equiv 1 \pmod{n}$$

even though n is not prime. This means Carmichael numbers are Fermat Pseudoprimes to all bases coprime to n and can fool Fermat Primality tests.

Necessary and Sufficient conditions (Korset's criterion)

A composite number n is a Carmichael number if and only if all the following hold:

1. n is square-free, i.e., no prime factor appears with power greater than 1.
2. n is composite.
3. For every prime divisor p of n , $p-1$ divides $n-1$.

Formally,

For all Primes $p|n$, $p-1|n-1$

Verification of $n = 561, 1105, 1729$

1. $n = 561$

• Prime factorization: $561 = 3 \times 11 \times 17$

• Square free? Yes

• Check divisibility:

⇒ $3-1 = 2 \mid 560$? Yes, Since $560/2 = 280$

⇒ $11-1 = 10 \mid 560$? Yes, Since $560/10 = 56$

⇒ $17-1 = 16 \mid 560$? Yes " $560/16 = 35$

• Conclusion: 561 is a Carmichael number.

2. $n = 1105$

⇒ Prime factorization = $5 \times 13 \times 17$

⇒ Square free? Yes

⇒ Check divisibility

• $5-1 = 4 \mid 1104$

• $13-1 = 12 \mid 1104$

• $17-1 = 16 \mid 1104$

∴ 1105 is a Carmichael number

3. $n = 1729$

⇒ Prime factorization = $7 \times 13 \times 19$

⇒ Square free? Yes

⇒ Check divisibility:

• $7-1 = 6 \mid 1728$

• $13-1 = 12 \mid 1728$

• $19-1 = 18 \mid 1728$

∴ 1729 is a Carmichael number.

Question-7: Determine whether the following are valid algebraic structures and justify your answer:

- i. Is the set $\mathbb{Z}_{11}$ with operations $(+, \cdot)$ a ring?
- ii. As the sets $(\mathbb{Z}_{37}, +)$ and $(\mathbb{Z}_{35}, \times)$ Abelian group?

Ans:

i) We check the ring axioms:

Additive Structure $(\mathbb{Z}_{11}, +)$:

- ⇒ Closure: The sum of any two integers mod 11 is still in $\mathbb{Z}_{11}$.
 - ⇒ Associativity: $(a+b)+c \equiv a+(b+c) \pmod{11}$.
 - ⇒ Identity: 0 is the additive identity.
 - ⇒ Inverses: For any a , the additive inverse is $11-a \pmod{11}$.
 - ⇒ Commutativity: $a+b \equiv b+a$.
- ∴ $(\mathbb{Z}_{11}, +)$ is an abelian group.

Multiplicative Structure $(\mathbb{Z}_{11}, \cdot)$:

- ⇒ Closure: Product mod 11 is in $\mathbb{Z}_{11}$.
- ⇒ Associativity: $(a \cdot b) \cdot c \equiv a \cdot (b \cdot c) \pmod{11}$.
- ⇒ Distributivity: $a \cdot (b+c) \equiv a \cdot b + a \cdot c \pmod{11}$.
- ⇒ Identity: 1 is the multiplicative identity.

Since, 11 is Prime, there are no zero divisions and every nonzero element has a multiplicative inverse making $\mathbb{Z}_{11}$ not just a ring but a field.

2) $(\mathbb{Z}_{37}, +)$

⇒ Closure: Sum mod 37 stays in $\mathbb{Z}_{37}$.

⇒ Associativity: Addition mod 37 is associative.

⇒ Identity: 0 is the additive identity.

⇒ Inverses: Every a has inverse $37 - a$.

⇒ Commutativity: $(a+b) \equiv b+a$.

All group satisfied so, It's an abelian group.

$(\mathbb{Z}_{35}, \times)$

⇒ Closure; Associativity, Identity 1: These hold for multiplication mod 35.

⇒ Inverses: Not every element has a multiplicative inverse.

Example: $5 \in \mathbb{Z}_{35}$ is a zero divisor Since $5 \cdot 7 \equiv 0 \pmod{35}$

If a shares a common factor with 35, it has no inverse mod 35.

⇒ Commutativity: Holds, but the lack of inverses means it's not a group.

∴ $(\mathbb{Z}_{35}, \times)$ is not an abelian group.

Problem-8: What is the remainder when -52 is reduced modulo 31 ?

Ans:

To find the remainder when -52 is reduced modulo 31 , we want a number r such that:

$$-52 \equiv r \pmod{31}$$

and $0 \leq r < 31$

Step-1: Find the positive equivalent of $-52 \pmod{31}$.

We can add multiples of 31 to -52 until the result is in the range $[0, 30]$:

$$-52 + 31 = -21 \quad (\text{still negative})$$

$$-21 + 31 = 10 \quad (\text{in the range } 0 \leq 10 < 31)$$

$\therefore$ Answer = 10 .

Problem-9: Determine the multiplicative inverse of 7 mod 26 if it exists. (Use extended Euclidean algorithm).

Ans:

We want n such that,

$$7n \equiv 1 \pmod{26}$$

Here 7 is already positive, so no need to convert.

So: $7n \equiv 1 \pmod{26}$

Using Extended Euclidean Algorithm

$$26 = 3 \cdot 7 + 5$$

$$7 = 1 \cdot 5 + 2$$

$$5 = 2 \cdot 2 + 1$$

$$2 = 2 \cdot 1 + 0$$

Back Substitution:

From $5 = 26 - 3 \cdot 7$

From $2 = 7 - 1 \cdot 5$

From $1 = 5 - 2 \cdot 2$

$$1. \quad 2 = 7 - 1(26 - 3 \cdot 7) = 7 - 26 + 3 \cdot 7 = 4 \cdot 7 - 26$$

$$2. \quad 1 = (26 - 3 \cdot 7) - 2 \cdot (4 \cdot 7 - 26)$$

$$= 26 - 3 \cdot 7 - 8 \cdot 7 + 2 \cdot 26$$

$$= 3 \cdot 26 - 11 \cdot 7$$

Here,

$$1 = 3 \cdot 26 - 11 \cdot 7$$
$$-11 \cdot 7 \equiv 1 \pmod{26}$$

$$\Rightarrow x \equiv -11 \pmod{26} \equiv 15 \pmod{26}$$

$$\therefore 7^{-1} \pmod{26} = 15$$

Pb-10: Evaluate $(-8 \times 5) \pmod{17}$, and explain how to simplify negative modular multiplication.

Ans:

$$(-8 \times 5) \pmod{17}$$

Step-1: Multiply first

$$-8 \times 5 = -40$$

$$\Rightarrow -40 \pmod{17} \equiv 11 \pmod{17}$$

$$\therefore -40 \equiv 11 \pmod{17}$$

(Ans.)

Problem-11: State and Prove Bezout's Theorem. Use it to find the multiplicative inverse of 97 modulo 385.

Ans:

Statement: For any integers a and b , there exist integers x and y such that,

$$ax + by = \gcd(a, b)$$

Proof:

Let $S = \{ax + by : x, y \in \mathbb{Z}, ax + by > 0\}$. S is a nonempty subset of positive integers. So it has a least element d by the well-ordering principle. We show $d = \gcd(a, b)$.

1. Because $d \in S$, there exist integers x_0, y_0 with

$$d = ax_0 + by_0$$

2. By the division algorithm divide a by d :

$$a = qd + r \text{ with } 0 \leq r < d. \text{ But } r = a - qd =$$

$$= a - q(ax_0 + by_0)$$

$$= a(1 - qx_0) + b(-qy_0), \text{ So } r \text{ is also an integer}$$

linear combination of a and b . If $r > 0$ then $r \in S$

contradicts minimality of d . So, $r = 0$. Thus $d \mid a$.

Similarly divide b by d to see $d \mid b$.

Hence d is a common divisor of a and b .

3. If c is any common divisor of a and b , then c divides every linear combination $ax+by$, so $c|d$. Thus d is the gcd. Therefore $d = \gcd(a, b)$, and the representation $ax_0 + by_0 = d$ is exactly Bezout's identity.

This Proves the theorem.

Problem-12: Using Bezout's identity find a n such that $43n \equiv 1 \pmod{240}$.

Ans:

We need integers n, y with

$$43n + 240y = 1.$$

Apply the Euclidean algorithm to 240 and 43.

$$240 = 5 \cdot 43 + 25,$$

$$43 = 1 \cdot 25 + 18,$$

$$25 = 1 \cdot 18 + 7$$

$$18 = 2 \cdot 7 + 4$$

$$7 = 1 \cdot 4 + 3$$

$$4 = 1 \cdot 3 + 1$$

$$3 = 3 \cdot 1 + 0$$

So, $\gcd(43, 240) = 1$ and we can back substitute to get Bezout coefficient. It has to be 1 and not 0.

$$1) \quad 1 = 4 - 1 \cdot 3$$

$$2) \quad 3 = 7 - 1 \cdot 4$$

$$\Rightarrow 1 = 4 - (7 - 4) = 2 \cdot 4 - 1 \cdot 7$$

$$3) \quad 4 = 18 - 2 \cdot 7$$

$$\Rightarrow 1 = 2(18 - 2 \cdot 7) - 1 \cdot 7 = 2 \cdot 18 - 5 \cdot 7$$

$$4) \quad 7 = 25 - 1 \cdot 18$$

$$\Rightarrow 1 = 2 \cdot 18 - 5(25 - 18) = 7 \cdot 18 - 5 \cdot 25$$

$$5) \quad 18 = 43 - 1 \cdot 25$$

$$\Rightarrow 1 = 7(43 - 25) - 5 \cdot 25 = 7 \cdot 43 - 12 \cdot 25$$

$$6) \quad 25 = 240 - 5 \cdot 43$$

$$\Rightarrow 1 = 7 \cdot 43 - 12(240 - 5 \cdot 43)$$

$$= 7 \cdot 43 - 12 \cdot 240 + 60 \cdot 43$$

$$= (7 + 60) \cdot 43 - 12 \cdot 240$$

$$= 67 \cdot 43 - 12 \cdot 240$$

Thus we have,

$$67 \cdot 43 + (-12) \cdot 240 = 1$$

$$\therefore 43^{-1} \bmod 240 = 67$$

Problem-13: Prove Fermat's Little Theorem and explain how it is used to test for primality. Is 561 a Prime number based on this test?

Ans:

Statement: If p is a prime number and a is an integer such that $\gcd(a, p) = 1$, then

$$a^{p-1} \equiv 1 \pmod{p}$$

Proof (Multiplicative Property Method)

1. Consider that

$$S = \{1, 2, 3, \dots, p-1\}$$

All elements are nonzero residues modulo p .

2. Multiply each element of S by a (where $\gcd(a, p) = 1$):

$$aS = \{a \cdot 1, a \cdot 2, \dots, a \cdot (p-1)\}$$

3. Since a is a coprime to p , multiplication by a is a permutation of S modulo p .

Therefore, aS contains exactly the same numbers as S , but in a different order.

Taking the Product of all elements in S:

$$1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-1) \equiv (a \cdot 1)(a \cdot 2) \cdot \dots \cdot (a \cdot (p-1)) \pmod{p}$$

$$(p-1)! \equiv a^{p-1} \cdot (p-1)! \pmod{p}$$

5. Since $(p-1)!$ is nonzero modulo p (because p is a prime), we can cancel it from both sides.

$$1 \equiv a^{p-1} \pmod{p}$$

Use for Primality Testing

• If p is Prime, for any a such that $\gcd(a, p) = 1$,

$$a^{p-1} \equiv 1 \pmod{p}$$

• If this fails for some a , then p is composite

• However, there are Carmichael numbers (like 561) that pass the test for all a but are not Prime.

Is 561 Prime?

We test with $a = 2$:

$\Rightarrow 561 = 3 \times 187 = 3 \times 11 \times 17 \rightarrow$ already composite, but let's

check Fermat's test

$\Rightarrow$ Compute: $2^{560} \pmod{561}$

Using modular exponentiation, it turns out:

$$2^{561} \equiv 1 \pmod{561}$$

Fermat's test says "maybe prime", but since we know

$$561 = 3 \times 11 \times 17,$$

It is not prime - it's a Carmichael number.

Problem-14: State and Prove the Chinese Remainder Theorem. Then solve the following system of congruences:

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}$$

Statement:

Let $m_1, m_2, \dots, m_k$ be pairwise coprime positive integers (i.e., $\gcd(m_i, m_j) = 1$ for $i \neq j$). Then

for any integer $a_1, a_2, a_3, \dots, a_k$, the system of congruences:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$\vdots$

$$x \equiv a_k \pmod{m_k}$$

has a unique solution modulo $M = m_1 m_2 \dots m_k$.

Proof of CRT.

We construct the solution explicitly

1. Compute $M = m_1 \cdot m_2 \cdot m_3 \cdots m_k$
2. for each i , compute $M_i = \frac{M}{m_i}$. Since the m_i 's are pairwise coprime, $\gcd(M_i, m_i) = 1$.
3. Find the modular inverse y_i of M_i modulo m_i , i.e.,
$$M_i y_i \equiv 1 \pmod{m_i}$$
4. The solution is:
$$x \equiv a_1 M_1 y_1 + a_2 M_2 y_2 + \cdots + a_k M_k y_k \pmod{M}$$

Solving the Given System:

We solve:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

Step-1: The moduli 3, 5, 7 are pairwise coprime.

Step-2: Compute $M = 3 \times 5 \times 7 = 105$

Step-3: Compute $M_1 = \frac{105}{3} = 35$,

$$M_2 = \frac{105}{5} = 21$$

$$M_3 = \frac{105}{7} = 15$$

Step-4: Find inverse

$$\Rightarrow y_1: \text{Solve } 35y_1 \equiv 1 \pmod{3}$$

$$35 \equiv 2 \pmod{3}$$

$$\text{So, } 2y_1 \equiv 1 \pmod{3}$$

The inverse is $y_1 = 2$ (Since $2 \times 2 = 4 \equiv 1 \pmod{3}$).

$$\Rightarrow y_2: \text{Solve } 21y_2 \equiv 1 \pmod{5}$$

$$21 \equiv 1 \pmod{5}, \text{ So } y_2 = 1$$

$$\Rightarrow y_3: \text{Solve } 15y_3 \equiv 1 \pmod{7}$$

$$15 \equiv 1 \pmod{7}, \text{ So } y_3 = 1$$

Step-5:

Compute the solution:

$$x \equiv 2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1 \pmod{105}$$

$$x \equiv 140 + 63 + 30 \pmod{105}$$

$$x \equiv 233 \pmod{105}$$

$$233 \pmod{105} = 23 \text{ (Since } 105 \times 2 = 210, \text{ and } 233 - 210 = 23)$$

$\therefore$ Final answer = 23.

Problem-15: Briefly explain the CIA triad in information Security. How does each component contribute to building a security system?

Ans: CIA Triad in Information Security refers to the three core principles for protecting information:

1. Confidentiality:

- ⇒ Meaning: Ensures that information is accessible only to authorized users.
- ⇒ Purpose: Prevents unauthorized disclosure of sensitive data.
- ⇒ Example: Encryption, access control lists, Strong passwords.

2. Integrity:

- ⇒ Meaning: Ensures that data is accurate, complete, and has not been altered without authorization.
- ⇒ Purpose: Maintains trustworthiness of information.
- ⇒ Example: Checksums, digital Signatures, version control.

3. Availability:

- ⇒ Meaning: Ensures that authorized users have timely and reliable access to information and systems.

- ⇒ Purpose: Prevents disruption of Service.
- ⇒ Example: Backups, redundancy, disaster recovery planning.

How each contributes to a secure system?

- ⇒ Confidentiality protects against data leakage and espionage.
- ⇒ Integrity ensures decisions are based on correct, unaltered data.
- ⇒ Availability ensures the system is usable when needed, preventing downtime from attacks.

Problem-16: How does steganography differ from cryptography in the context of information security, and what are common techniques used for hiding data in digital media?

Ans:

Difference between Steganography and cryptography

⇒ Cryptography: Focuses on scrambling the content of a message so it becomes unreadable to unauthorized users.

- Even if intercepted, the message looks like random text unless the key is known.

- Example: AES encryption.

⇒ Steganography: Focuses on hiding the existence of the message itself by embedding it into another medium.

- The message does not attract attention, as it appears like a normal file.

- Example: hiding text inside an image file's pixel values.

Common Techniques for Hiding Data in Digital Media:

1. Least Significant Bit (LSB) Insertion

⇒ At least significant bits of pixels or audio samples to store hidden data without noticeably changing quality.

2. Masking and Filtering

⇒ Embed information in significant areas of the file so changes blend with original content.

3. Transform Domain Technique

⇒ Hide data in transformed coefficient.

4. Spread Spectrum

⇒ Spread hidden data across a wide frequency spectrum to make detection harder.

5. Metadata Manipulation

⇒ Store hidden message in unused or non-critical metadata fields of image, audio, or video files.

Problem-17: What are the key differences between phishing, malware, and denial-of-services (DoS) attack in terms of their method and impact on system security?

Ans:

Phishing:

- i) Method: Social engineering - attackers send deceptive emails/messages or create fake websites to trick users into revealing credentials, personal data or clicking malicious links.
- ii) Goal/Impact: Credential theft, identity theft, initial foothold in networks.
- iii) Defences: User education, email filtering, multi factor authentication (MFA), URL/website scanners, anti phishing policies.

Malware:

- i) Method: Software (viruses, worms, trojans, ransomware, spyware) delivered via email attachments, drive by downloads, infected devices or malicious installers.

i) Goal/Impact: Data theft, destruction, system compromise, encryption for ransom establishing persistent backdoors.

Denial-of-Service (Dos / DDos)

i) Method: Overwhelm a service or network with traffic or exploit resource limits.

ii) Goal/Impact: Availability loss - legitimate users cannot access service, can cause business disruption and revenue loss.

iii) Defence: Rate limiting, traffic filtering, DDos mitigation services (scrubbing), content delivery networks, redundancy.

Problem-18: Explain how legal frameworks such as the General Data Protection Regulation (GDPR) help mitigate cyber attacks and protect user privacy.

Ans:

The General Data Protection Regulation (GDPR) is a legal framework introduced by the European Union to strengthen the protection of personal data and privacy. It helps mitigate cyber attacks and protect user privacy in several ways.

1. Data minimization and Security requirements: Organizations must collect only the necessary personal data and implement strong security measures to protect it from breaches.
2. Breach Notification: GDPR mandates that data breaches must be reported to authorities and, in some cases, to affected individuals within 72 hours, enabling faster response and mitigation.
3. Accountability and penalties: Organizations are held accountable for compliance, and non-compliance can result in heavy fines, which encourage companies to prioritize cyber security and privacy.

Question -19: Explain the basic working of the DES algorithm using a simple 64-bit plaintext block and a 56-bit key. show how the initial permutation, round functions, and final permutation contribute to the encryption process.

Ans:

Des Algorithm Summary:

Input:

- 64-bit plaintext block
- 56-bit key

Steps:

1. Initial Permutation (IP)
2. 16 Rounds of Feistel Network
3. Final Permutation (IP^{-1})

1. Initial Permutation (IP):

The plaintext is rearranged using a fixed IP table (given in exams).

Example:

plaintext = 0000 0001 0011 ... 1111 (64 bits)

After IP = 1100 1010 ... (shuffled bits)

2. Key Schedule (56 bit $\rightarrow$ 48 bit Round Key)

1. PC-1 Permutation:

- The 56-bit key is permuted (discarding Parity bits)
- Output: Two 28 bits halves (C_0 and D_0).

2. Left Shift and PC-2:

- For each round, C_n and D_n are left-shifted
- PC-2 compresses 56 bits $\rightarrow$ 48-bit round key.

3. Feistel Round (16 Rounds)

Each round Process 32-bit halves

Round Function (F):

1. Expansion (E-box):

- Expands R_n (32-bit) $\rightarrow$ 48-bit

2. XOR with Round key (K_n):

- $E(R_n) \oplus K_n \rightarrow$ 48 bit intermediate

3. S-box Substitution (6 bit $\rightarrow$ 4 bit)

- Split into 8 chunks of 6 bits $\rightarrow$ output: 4 bits per chunk.

4. P-box permutation:

- Shuffles the 32 bit output.

Fistel Swap:

• New left half: $L_{n+1} = R_n$

• New right half: $R_{n+1} = L_n \oplus F(R_n, K_n)$

• Final Round: No Swap!

4. Final Permutation (IP^{-1})

• After 16 rounds, apply IP^{-1} to get ciphertext.

Problem-20: In the DES algorithm, a 64-bit plaintext block is divided into two 32-bit halves: L_0 and R_0 . Given $R_0 = 0xFOFOFOFO$ and the round key $K_1 = 0x0F0F0F0F$, compute the output of the first round's function $f(R_0, K_1)$ assuming XOR operation only.

Ans:

Given:

$$R_0 = 0xFOFOFOFO$$

$$\text{Round key } K_1 = 0x0F0F0F0F$$

$$L_0 = 0xAAAAAAAA$$

Step 1: Compute $f(R_0, K_1)$ assuming XOR operation only.

$$f(R_0, K_1) = R_0 \oplus K_1$$

$$= 0xFOFOFOFO \oplus 0x0F0F0F0F$$

Perform XOR byte-wise:

- $FO \oplus 0F = FF$
- $FO \oplus 0F = FF$
- $FO \oplus 0F = FF$
- $FO \oplus 0F = FF$

$$\text{So, } f(R_0, K_1) = 0xFFFF.FFFF$$

Step-2: Compute $L_1 = R_0 = 0x\text{F0F0F0F0}$

Step-3: Compute $R_1 = L_0 \oplus f(R_0, k_1)$

$$\Rightarrow R_1 = 0x\text{AAAAAAAA} \oplus 0x\text{FFFFFFFF}$$

XOR byte-wise:

$$\bullet AA \oplus FF = 55$$

$$\bullet AA \oplus FF = 55$$

$$\bullet AA \oplus FF = 55$$

$$\bullet AA \oplus FF = 55$$

$$\text{So, } R_1 = 0x\text{55555555}$$

$$\therefore L_1 = 0x\text{F0F0F0F0}, R_1 = 0x\text{55555555}$$

Question-21: Given the input word:

$[0x23, 0xA7, 0x4C, 0x19]$

Use the following partial AES S-box to perform the SubBytes transformation. Provide the resulting output word.

Row $\downarrow$ \ Col $\rightarrow$ 3 4 7 9 A C

1	6D	.	.	C6	.	.
2	D4	.	.	.	.	.
4	.	A ₁	.	.	.	2E
A	.	.	63	.	D ₂	.

Ans: $[0x20, 0x20, 0x20, 0x20]$

Given the input word:

$[0x23, 0xA7, 0x4C, 0x19]$

For each byte, the high nibble (4 byte bits) is the row, and the low nibble is the column.

Let's find the corresponding S-box value for each byte:

- For $0x23$: row = 2, col = 3 $\rightarrow$ from table,
row 2 col 3 = D₄

• For $0xA7$: row = A, col = 7 $\rightarrow$ from table,
row A col 7 = 63

• For $0x4C$: row = 4, col = C $\rightarrow$ from table,
row 4 col C = not given

So, we leave it as is or unknown.

• For $0x19$: row = 1, col = 9 $\rightarrow$ from table
row 1 col 9 = C6

Resulting output word:

$[0xD4, 0x63, \text{unknown}, 0xC6]$

Question-22: In AES encryption, apply the AddRoundKey step only. Given:

- Input word: $[0x1A, 0x2B, 0x3C, 0x4D]$
- Round key word: $[0x55, 0x66, 0x77, 0x88]$

Perform XOR between the input word and the round key, and write the resulting output word.

Ans: The AddRoundKey step in AES encryption involve performing a Bitwise XOR operation between the input word and the round key word.

Given:

input word: $[0x1A, 0x2B, 0x3C, 0x4D]$

Round key word: $[0x55, 0x66, 0x77, 0x88]$

Here,

$$0x1A \oplus 0x55 = 0x4F \quad (00011010 \oplus 01010101) \\ = 01001111$$

$$0x2B \oplus 0x66 = 0x4D \quad (00101011 \oplus 01100110 = 01001101)$$

$$0x3C \oplus 0x77 = 0x4B \quad (00111100 \oplus 01110111 = 01001011)$$

$$0x4D \oplus 0x88 = 0xC5 \quad (01001101 \oplus 10001000 = 11000101)$$

Resulting output word: $0x4B$

$[0x4F, 0x4D, 0x4B, 0xC5]$

Brief explanation: The AddRoundKey step performs a bitwise XOR between each byte of the state and the corresponding byte of the round key, producing the output word shown above.

Input word: $0x4F, 0x4D, 0x4B, 0xC5$

Round key word: $0x77, 0x88, 0x99, 0xA0$

$$(01001111 \oplus 01110111) \oplus 0x4F = 0x4B \quad (01001111 \oplus 01110111 = 00111000)$$

$$(01001101 \oplus 10001000) \oplus 0x4D = 0xC5 \quad (01001101 \oplus 10001000 = 11000101)$$

Question-23: Show how MixColumns uses the following fixed matrix over $GF(2^8)$:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

to transform an input column. Use an example column with values $[0x01, 0x02, 0x03, 0x04]$. (Convert Hex to binary)

Ans:

The MixColumns operation in AES works by multiplying each column of the state matrix by a fixed matrix over the Galois field $(GF) : 2^8$.

Given that,

Input column (bytes):

$$a = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0x01 \\ 0x02 \\ 0x03 \\ 0x04 \end{bmatrix}$$

Mixcolumns Matrix:

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

So, the output column is $b = M \cdot a$.

Where each entry is bitwise ($\text{GF}(2^8)$) arithmetic (XOR for addition).

10	10	50	50
10	50	50	10
50	50	10	10
50	10	10	50

Row 1 calculation:

$$b_0 = (0_2 \cdot a_0) \oplus (0_3 \cdot a_1) \oplus (0_1 \cdot a_2) \oplus (0_1 \cdot a_3)$$

Compute terms:

$$\rightarrow 0_2 \cdot a_0 = 0_2 \cdot 0x01 = \text{xtime}(0x01) = 0x02$$

$$\rightarrow 0_3 \cdot a_1 = (0_2 \cdot 0x02) = \text{xtime}(0x02) \text{ XOR } 0x02$$

$$0x02 = 0x04 \text{ XOR } 0x02$$

$$= 0x06$$

$$\rightarrow 0_1 \cdot a_2 = 0x03$$

$$\rightarrow 0_1 \cdot a_3 = 0x04$$

Now XOR them:

$$b_0 = 0x02 \oplus 0x06 \oplus 0x04 \oplus 0x03$$

$$= 0x03$$

$$\text{So, } b_0 = 0x03$$

10	10	50	50
10	50	50	10
50	50	10	10
50	10	10	50

$= M$

Similarly,

Final result (Output column)

$$b = \begin{bmatrix} 0x03 \\ 0x04 \\ 0x09 \\ 0x0A \end{bmatrix}$$

Binary form -

$$\cdot 0x03 = 00000011$$

$$\cdot 0x04 = 00000100$$

$$\cdot 0x09 = 00001001$$

$$\cdot 0x0A = 00001010$$

Question - 24: Describe How AES-OFB mode works. How does it ensure Synchronization between encryption and decryption streams?

Ans:

AES-OFB (Output feedback) is a mode of operation for the Advanced Encryption Standard (AES) that turns a block cipher into a stream cipher. It works by generating a keystream, which is then XORed with the plaintext to produce ciphertext.

How it works:

1. Initialization: The process starts with a unique initialization vector (IV). Which is typically a random value. This IV is fed into the AES block cipher.
2. Keystream Generation: The output of the AES encryption of the IV becomes the first part of the keystream. This output is also feedback into the AES block cipher for the next round, generating the next part of the keystream.

3. Encryption: Each block of the keystream is XORed with corresponding block of plaintext to produce the ciphertext.

Synchronization in AES-OFB:

- Both sender and receiver start with the same Initialization vector (IV) and same secret key.
- Keystream blocks are generated only from the previous keystream block, not from the ciphertext or plaintext.
- This means if IV and key are identical, both sides produce exactly the same keystream sequence, keeping encryption and decryption aligned.

Question-25: Which AES modes cause error propagation during decryption, and how does it affect the integrity of the decrypted message?

Ans:

AES modes causing Error Propagation:

Some AES modes (CBC and CFB) cause an error in a ciphertext block to affect multiple plaintext blocks during decryption.

This reduces the integrity of the decrypttext message because even a single bit error can corrupt more than one block.

1. CBC (Cipher Block Chaining) Mode:

- Decryption formula:

$$P_i = AES^{-1}(C_i) \oplus C_{i-1}$$

- Error effect:

⇒ If C_i has a 1-bit error → P_i becomes completely random.

⇒ The same bit position in P_{i+1} will

also be flipped.

2. CFB (cipher feedback) Mode:

⇒ Decryption formula:

$$P_i = C_i \oplus \text{AES}(C_{i-1})$$

⇒ Error effect:

- If C_i has a 1-bit error → the same bit P_i is wrong.
- P_{i+1} becomes completely random.
- After that, decryption returns to normal.

Problem-26: Which AES mode would you recommend for encrypting large files with parallel processing, and why? Justify your choice between ECB, CBC, and CTR.

Ans:

Recommended Modes: AES-CTR (Counter Mode).

Justification:

- Parallel Processing:

⇒ In CTR, each block is encrypted by XORing the plaintext with a keystream generated from AES.

⇒ The counter values for all blocks are known in advance, so encryption and decryption of different blocks can happen independently and in parallel.

- Performance:

⇒ No chaining between blocks - faster than CBC for large files.

- Security:

⇒ Unlike ECB, CTR does not produce identical ciphertext for identical plaintext blocks.

⇒ As secure as CBC when IV/nonce is unique.

Flexibility:
⇒ Works efficiently for both stream-like and random access data.

Why not ECB?

Reveals patterns in the data - insecure for large bits.

Why not CBC?

Each block depends on the previous ciphertext block cannot be parallelized in encryption.

For large file encryption with parallel processing, AES-CTR is best because it is secure and highly parallelizable.

Problem-27: Given a message "A" represented as $M=1$, encrypt it using a public key $e=5$, $n=14$. What is the ciphertext? Then decrypt using Private key $d=11$.

Solution:

Given,

$$M=1, e=5, n=14, d=11.$$

Encryption: $C \equiv M^e \pmod{n} = 1^5 \pmod{14} = 1$

Decryption: $M \equiv C^d \pmod{n} = 1^{11} \pmod{14} = 1$

Why this works:

RSA relies on $M^{ed} \equiv M \pmod{n}$ where $ed \equiv 1 \pmod{\phi(n)}$.

Here, $\phi(14) = 6$ and $e \cdot d = 5 \cdot 11 = 55 \equiv 1 \pmod{6}$.

So, decryption recovers M .

$\therefore$ Ciphertext $C=1$ and Decrypted Message, $M=1$.

Problem-28: Given Message hash: $H(M) = 5$,
RSA Private key $d = 3$, $n = 33$. Generate
the digital Signature.

Ans:

Given: hash $H(M) = 5$, $d = 3$, $n = 33$.

Signature generation (Sign with private keys)

We need to generate a digital signature for
a message hash.

The digital signature S is computed using the
formula:

$$S' = H(M)^d \pmod{n}$$

$$S = 5^3 \pmod{33}$$

$$= 125 \pmod{33}$$

$$\therefore 125 = 3 \times 33 + 26,$$

$$\text{So, } 125 \pmod{33} = 26$$

$\therefore$ The digital signature is 26.

Problem-29: Aleya and Badol are using the Diffie-Hellman key exchange Protocol. They agree on the following Public values: Prime modulus $P=17$, Base $g=3$, Aleya's Private key $a=4$, Badol's Private key $b=5$.
Compute the public key of Aleya and Badol.

Solution:

We'll compute the public keys for Aleya and Badol based on the Diffie-Hellman Protocol.

- Public values: Prime modulus $P=17$,
base $g=3$.

- Aleya's Private key $a=4$

- Badol's Private key $b=5$

- Aleya's Public key(A):

$$\Rightarrow A = g^a \pmod{P} = 3^4 \pmod{17} = 81 \pmod{17}$$

$$\Rightarrow 81 = 4 \times 17 + 13.$$

$$\text{So, } 81 \pmod{17} = 13$$

$\Rightarrow$ Aleya's public key is 13.

• Badol's public key (B):

$$\Rightarrow B = g^b \pmod{p} = 3^5 \pmod{17} = 243 \pmod{17}.$$

$$\Rightarrow 243 = 14 \times 17 + 5$$

$$\text{So } 243 \pmod{17} = 5$$

$\Rightarrow$ Badol's public key is 5.

Problem-30:

$$H(m) = \left(\sum \text{ASCII of character in } m \right) \pmod{100}$$

compute the hash value of the message

"AB" and "BA" using this function. What

does this imply about collision resistance

in weak hash function?

Ans:

Definition:

$$H(m) = \left(\sum \text{ASCII (chars in } m) \right) \pmod{100}$$

ASCII values: A = 65, B = 66

compute hashes:

$$\begin{aligned} H("AB") &= (65 + 66) \pmod{100} \\ &= 131 \pmod{100} \\ &= 31 \end{aligned}$$

$$\begin{aligned}
 H("BA") &= (66 + 65) \bmod 100 \\
 &= 131 \bmod 100 \\
 &= 31
 \end{aligned}$$

Result:

Both "AB" and "BA" produce the same hash $\Rightarrow$ collision.

Collision Resistance:

$\Rightarrow$ The two different messages, "AB" and "BA" produce the same hash value which is called collision.

$\Rightarrow$ This hash function is not collision-resistant because it's based on a simple modular sum.

Problem-31: In a secure message system, a Simple Message Authentication Code (MAC) is computed using modular addition:

$$MAC = (\text{Message} + \text{Secret/Key}) \bmod 17.$$

Ans:

Given,

$$\text{Message } M=15, K=7.$$

Compute MAC:

$$MAC = (15 + 7) \bmod 17$$

$$= 22 \bmod 17$$

$$= 5$$

Attacker Scenario:

Suppose attacker changes message to $M'=10$ but does not know K .

Read MAC for M' is:

$$MAC' = (10 + 7) \bmod 17$$

$$= 17 \bmod 17$$

$$= 0$$

Why forging is easy?

Because MAC is linear: $MAC = M + K \pmod{17}$.

If attacker knows one valid pair (M, MAC) , and wants to create $M' = M + \Delta$, they can compute

$$MAC' = MAC + \Delta \pmod{17} \text{ without knowing } K.$$

Example: here,

$$\Delta = -5 \equiv 12 \pmod{17};$$

$$MAC' = 5 + 12 = 17 \equiv 0, \text{ which matches the true MAC.}$$

Conclusion: This MAC is insecure it is trivially forgeable because it leaks linear relation with the message.

Problem-32: Explain the steps involved in the TLS handshake process. How are symmetric keys established securely using asymmetric cryptography during the handshake?

Ans:

TLS handshake steps & symmetric key Establishment:

1. ClientHello: Client sends supported TLS version, cipher suites, random number.
2. ServerHello: Server selects TLS version and cipher suite, sends its random number.
3. Server Certificate: Contains server's public key, signed by CA for Authentication.
4. Key Exchange:
 - RSA mode
 - ECDHE / DHE mode.
5. Finished Message: Both parties send encrypted "Finished" message to confirm the handshake.

How Symmetric keys are established Secure?

Asymmetric encryption (RSA or Diffie-Hellman) ensures that only the intended parties can compute the shared secret from which symmetric keys are derived.

Problem-33: Explain the layered architecture (Protocol Stack) of SSH. Briefly describe the roles of each layer.

Ans:

SSH refers to "secure shell" is a protocol that provides a secure channel over an unsecured network. It has a layered architecture consisting of three main layers.

1. Transport layer Protocol:

This is the lowest layer, responsible for managing the secure connection.

- Handles encryption
- Integrity Protection
- Compression
- Server authentication

2. User Authentication Protocol:

This layer runs on top of the transport layer and handles client authentication.

- Verifies the client's identity using password.
- Public keys or other methods.

3. Connection Protocol:

Multiplexes the encrypted channel into multiple logical channels. This is the highest layer.

Question-34: Explain the steps involved in the TLS handshake process.

Ans:

The steps involved in the TLS handshake process are given below:

1. ClientHello: It's a process connection parameters.
2. ServerHello: It's also a proposes connection.

Parameters.

3. Certificate key exchange

4. Generate Master Secret

5. Generate Session keys.

6. Finished Messages

7. Secure Communication.

Question-35: What is the general form of all elliptic curve equation over a finite field, and why is it used in cryptography?

Ans:

The general form of an elliptic curve equation over a finite field is:

$$y^2 = x^3 + ax + b \pmod{p}.$$

Here,

p is a large Prime number that defines the finite fields and a and b are constants such that,

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}$$

Uses is cryptography:

Provides a group structure for elliptic curve cryptography (ECC), enabling secure key exchange, signature and encryption with smaller keys.

Question-36: How does ECC achieve the same level of security as RSA with a smaller key size? Briefly explain.

Ans:

Elliptic curve cryptography (ECC) is a public key cryptography technique that provides the same cryptographic strength as RSA but with much smaller key sizes. The mathematical problems.

1. Mathematical Foundation:

- ⇒ RSA is based on the integer factorization problem.
- ⇒ ECC is based on the elliptic curve discrete logarithm problem.

2. Key Size Comparison:

⇒ ECC 256-bit $\approx$ RSA 3072-bit Security level.

⇒ ECC 384-bit $\approx$ RSA 7680-bit Security level.

3. Conclusion:

ECC Provides Strong Security with reduced key sizes due to the computational hardness of the EC DLP compared to integer factorization.

Question-37:

Given the elliptic curve $y^2 = x^3 + 2x + 3 \pmod{37}$.

Determine whether the point $P = (3, 6)$ lies on the curve.

Ans:

Given that,

curve:

$$y^2 \equiv x^3 + 2x + 3 \pmod{37}$$

point: $P = (3, 6)$

$$\text{L.H.S} = y^2 = 6^2 = 36 \pmod{37}$$

$$RHS = n^2 + 2n + 3 = 27 + 6 + 3 = 36 \pmod{97}$$

Since, $LHS = RHS$.

$\therefore$ point lies on the curve.

Question-38: Given public key $(p=23, g=5, h=8)$ and message $m=10$, compute the ElGamal ciphertext using random $k=6$.

Ans:

To compute the ElGamal ciphertext, we use the following public values:

$p=23, g=5, h=8$ and the message

$m=10$, The random number is

$$k=6$$

The ciphertext consist of two points (c_1, c_2)

Here,

$$c_1 = g^k \pmod{p}$$

$$\Rightarrow c_1 = 5^6 \pmod{23}$$

$$\Rightarrow 5^v = 25 \equiv 2 \pmod{23}$$

$$\Rightarrow 5^6 = 5^4 \times 5^v \equiv 4 \times 2 = 8 \pmod{23}$$

$$\text{So, } c_1 = 8.$$

$$\text{Again, } c_2 = m \times h^k \pmod{p}$$

First we calculate h^k :

$$h^k = 8^6 \pmod{23}$$

$$\Rightarrow 8^v = 64 = 2 \times 23 + 18 \equiv 18 \pmod{23}$$

$$\Rightarrow 8^6 = 8^4 \times 8^v \equiv 2 \times 18 = 36 = 1 \times 23 + 13 \equiv 13 \pmod{23}$$

Now,
we compute c_2 :

$$c_2 = 10 \times 13 \pmod{23} = 130 \pmod{23}$$

$$\Rightarrow 130 = 5 \times 23 + 15 \equiv 15 \pmod{23}$$

$$\text{So, } c_2 = 15$$

The ElGamal Ciphertext is $(8, 15)$.

$$\boxed{c_1, c_2 = (8, 15)}$$

Question - 39:

Explain how lightweight cryptography is important for securing IOT devices. Give one example of a lightweight encryption algorithm used in IOT.

Ans:

Lightweight cryptography is specially designed to provide strong security while using minimal computational resources, memory and power. This is crucial for IOT (Internet of Things) devices, such as sensors, wearables and smart applications, which often have limited processing power, battery capacity, and storage. Traditional encryption algorithms like AES or RSA may be too heavy for these devices, causing delays or draining power. Lightweight cryptography ensures:

⇒ low power consumption - extends battery life of IOT devices.

$$(21, 8) = 2012$$

⇒ Low memory and CPU usage - runs efficiently on constrained hardware.

⇒ Adequate Security - Protects data and communication from attacks.

Example:

PRESENT cipher - a block cipher with 64-bit block size and 80/128 bit keys, designed for low-cost hardware. It is widely used in IOT.

Lightweight cryptography enables secure communication in IOT without overloading the devices limited resources.

Question - 40:

List and briefly explain any three common IOT-specific attacks (eg. firmware hijacking, physical tempering, botnets like Mirai).

What mitigation strategies can be applied?

Ans:

Three common IOT-specific attacks and mitigation strategies are given below:

1. Firmware Hijacking:

⇒ Explanation:

Attackers replace or modify device firmware with malicious code, allowing them to take control, steal data, or disrupt functionality.

⇒ Mitigation:

Use digitally signed firmware enable secure boot, and only allow updates from trusted servers.

2. Physical Tempering:

⇒ Explanation:

IOT devices in public or unprotected areas can be physically accessed. Attackers may open the device, extract stored data, or modify hardware.

⇒ Mitigation:

Use tamper resistant casings, disable debug ports and encrypt sensitive stored data.

3. Botnet Attacks:

⇒ Explanation:

Malware infects IoT devices with weak/default passwords, linking them into a botnet for large scale DDOS or spam attack.

⇒ Mitigation:

Change default credentials keep firmware updated, and use firewalls or intrusion detection systems.